

1 背景知识

1.1 项目基本介绍

TempleDAO 是一个基于以太坊的 DeFi 协议，旨在通过算法稳定币和质押机制提供可持续的收益。本次受影响的组件是 StaxLPStaking 合约，该合约的主要功能是允许用户质押流动性代币（LP Tokens）以获取协议奖励（TEMPLE 代币等）。该合约包含了一个为了方便旧版本迁移而设计的“迁移功能”，旨在让用户能平滑地将旧合约中的质押份额迁移至新合约。

1.2 受害者函数

漏洞存在于 StaxLPStaking.sol 合约的 migrateStake 函数中。该函数允许外部调用者指定一个“旧合约”地址，并请求将该地址下的资金迁移至当前合约。代码具体如下图 1 所示。

```
function migrateStake(address oldStaking, uint256 amount) external {  
    StaxLPStaking(oldStaking).migrateWithdraw(msg.sender, amount); // 外部调用  
    _applyStake(msg.sender, amount); // 状态更新  
}
```

图 1 受害者函数具体代码

1.3 攻击者地址

根据提供的交易哈希 0x8c3f442fc6d640a6ff3ea0b12be64f1d4609ea94edd2966f42c01cd9bdcf04b5，追踪到的链上实体如下：

攻击者合约：0x2Df9c154fe24D081cfE568645Fb4075d725431e0

受害者合约：0xd2869042E12a3506100af1D192b5b04D65137941

2 关键攻击路径

攻击者利用了合约对 oldStaking 参数缺乏验证的漏洞，执行了经典的重放/欺骗攻击。以下是交易的详细执行逻辑：

1. 攻击者首先部署了一个恶意的合约。该合约实现了一个虚假的 migrateWithdraw 函数，该函数在被调用时什么都不做，不报错且立即返回。
2. 攻击者调用受害合约的 migrateStake(address oldStaking, uint256 amount)函数。

可以看到，攻击者在调用受害合约时，参数 oldStaking: 填入攻击者刚刚部

署的假合约地址。参数 `amount`: 填入巨额数值，也是损失金额，为 321154865567124596801893。

3. 受害合约执行 `StaxLPStaking(oldStaking).migrateWithdraw(...)` 由于调用的是假合约，该步骤顺利通过，没有任何真实的资金从旧合约转入受害合约。

4. 受害合约继续执行 `_applyStake(msg.sender, amount)`，结果攻击者在并未存入任何资产的情况下，账面余额被增加了。

5. 攻击者立即调用 `withdrawAll` 函数，合约检查 `_balances`，确认攻击者有余额（尽管是凭空产生的），合约执行 `stakingToken.safeTransfer`，将其他真实用户存入的代币转账给攻击者。具体过程如下图 2 所示



3 资金流与净利润

本次攻击导致 TempleDAO 的 `StaxLPStaking` 池被耗尽，造成了显著的经济损失。

被盗资产： 321,154.865567124596801893 代币。

净利润估算： 1,261,929.5\$

Addresses	Token	TokenID	Balance	Value in USD	Total Value in USD
0x2df9c154fe24d081cfe568645fb4075d725431e0 [Receiver]	xFraxTempleLP	-	+321154.865567124596801893	\$1261929.5	+\$1261929.5
StaxLPStaking	xFraxTempleLP	-	-321154.865567124596801893	\$1261929.5	-\$1261929.5

图 3 资金估计

4 攻击根本成因

本次攻击的根本原因属于输入验证不当导致了严重的信任边界破坏，代码详细见图 1 所示。

1. 函数 `migrateStake` 允许任意用户传入 `oldStaking` 地址。开发者错误地假设用户只会传入合法的旧合约地址，而没有在代码层面强制验证该地址的合法性。

2. 记账操作 `_applyStake` 完全依赖于外部调用 `migrateWithdraw` 的成功返回，而不是依赖于实际收到的代币数量。正确逻辑应为：实际记账金额 = 迁移后余

额 - 迁移前余额，但开发者逻辑为：实际记账金额 = 用户声称的迁移金额。

5 防御建议措施

为了防止此类漏洞再次发生，建议采取以下修复措施：

1. 实施白名单机制

这是最直接有效的防御手段。对于涉及资金迁移的敏感函数，必须限制 `oldStaking` 参数只能是经过治理层投票确认的官方合约地址。

```
//添加白名单检查
mapping(address => bool) public isTrustedStakingContract;

function migrateStake(address oldStaking, uint256 amount) external {
    require(isTrustedStakingContract[oldStaking], "Error: Untrusted staking contract");
    StaxLPStaking(oldStaking).migrateWithdraw(msg.sender, amount);
    _applyStake(msg.sender, amount);
}
```

图 3 白名单机制参考代码

2. 采用“检查-生效-交互”模式的变体

不要信任外部调用的结果，而要信任合约自身的资产状态。在执行迁移前后进行余额快照对比。

```
//余额检查
function migrateStake(address oldStaking, uint256 amount) external {
    // 1. 记录迁移前的余额
    uint256 balanceBefore = stakingToken.balanceOf(address(this));

    // 2. 执行迁移
    StaxLPStaking(oldStaking).migrateWithdraw(msg.sender, amount);

    // 3. 记录迁移后的余额
    uint256 balanceAfter = stakingToken.balanceOf(address(this));

    // 4. 验证实际收到的金额是否匹配
    require(balanceAfter >= balanceBefore + amount, "Error: Fund transfer failed");

    // 5. 记账
    _applyStake(msg.sender, amount);
}
```

图 4 余额检查参考代码